

MongoDB Authentication Commands

Syntax:

1. `db.auth( "username", "password" )`

MongoDB employs a robust mechanism to control access and privileges to a databases system known as Role-Based Access Control (RBAC).

How MongoDB Controls Access with Role-Based Access Control

Access control — also known as authorization — is a security technique that involves determining who can gain access to which resources.

Authentication is the process of confirming whether a user or client is actually who they claim to be. Authorization, on the other hand, involves setting rules for a given user or group of users to define what actions they can perform and which resources they can access.

Authentication in MongoDB

In many database management systems, users are identified with just a username and password pair. When connecting to a database with valid credentials, the user is authenticated and granted the level of access associated with that user.

In contrast, MongoDB employs a more complex user directory structure. In MongoDB users are not only identified by their usernames, but also by the database in which they were created. For each user, the database in which they were created is known as that user's authentication database.

Authorization in MongoDB (Role-Based Access Control)

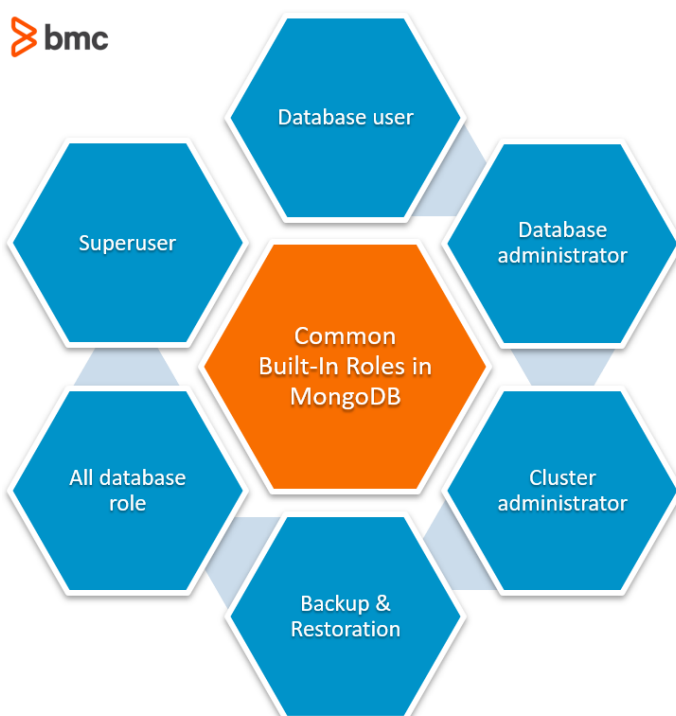
In MongoDB, you control who has access to what resources on a database and to which degree through a mechanism called Role-Based Access Control, often shortened as RBAC.

In Role-Based Access Control, users are not given permissions to perform actions on resources directly, such as inserting a new document into the database or querying a particular collection. This would make the security policies difficult to manage and keep consistent with many users in the system. Instead, the rules allowing actions on particular resources are assigned to roles.

There are two types of Roles in MongoDB:

- **Built-In Roles.** MongoDB provides built-in roles to offer a set of privileges that are commonly needed in a database system.
- **User-Defined Roles.** If built-in roles do not provide all the expected privileges, database administrators can define custom roles using the **createRole** method. Those roles are called User-Defined roles.

Next, let's look at the variety of roles in more details.



Built-in Roles

In this section, we'll go through the most common built-in roles of MongoDB.

Database user roles

Database user roles are normal user roles that are useful in regular database interactions.

Role	Description
read	Read all non-system collections and the system.js collection
read Write	Both Read and Write functionality on non-system collections and the system.js collection

Database administration roles

These are roles that are used to carry out administrative operations on databases.

Role	Description
dbAdmin	Perform administrative tasks such as indexing and gathering statistics, but cannot manage users or roles
userAdmin	Provides the ability to create and modify roles and users of a specific database
dbOwner	This is the owner of the database who can perform any action. It is equal to combining all the roles mentioned above: readWrite, dbAdmin, and userAdmin roles

Backup & restoration roles

These are the roles that are required for backup and restoring data in a MongoDB instance. They can only be assigned with the admin database.

Role	Description
backup	Provides the necessary privileges to backup data. This role is required for MongoDB Cloud Manager and Ops Manager, backup agents, and the mongodump utility.
restore	Provides the privileges to carry out restoration functions

All database roles

These are database roles that provide privileges to interact with all databases, excluding local and config databases.

Role	Description
readAnyDatabase	Read any database
readWriteAnyDatabase	Provides read and write privileges to all databases
userAdminAnyDatabase	Create and Modify users and roles across all databases
dbAdminAnyDatabase	Perform database administrative functions on all databases

Superuser roles

MongoDB can provide either direct or indirect system-wide superuser access. The following roles grant superuser privileges scoped to a specified database or databases.

- `dbOwner`
- `userAdmin`
- `userAdminAnyDatabase`

The true superuser role is the **root** role, which provides systemwide privileges for all functions and resources.

User-Defined Roles

In situations where built-in roles are unable to provide the necessary privileges covering the scope of the access requirements or to restrict the scope and actions of a user, you can define custom User-Defined roles.

MongoDB Role Management provides the necessary methods to create and manage user-defined roles. The most commonly used methods for user-defined role creation are shown in the following table. (A complete list of methods can be found [here](#).)

Method	Description
<code>db.createRole()</code>	Create a role and its privileges
<code>db.updateRole()</code>	Update the user-defined role
<code>db.dropRole()</code>	Delete a user-defined role
<code>db.grantPrivilegesToRole()</code>	Assigns new privileges to a role

```
db.revokePrivilegesFromRole()
```

Removes privileges from a role

Step 1 — Outlining the Example Scenario and Preparing the Sample Databases

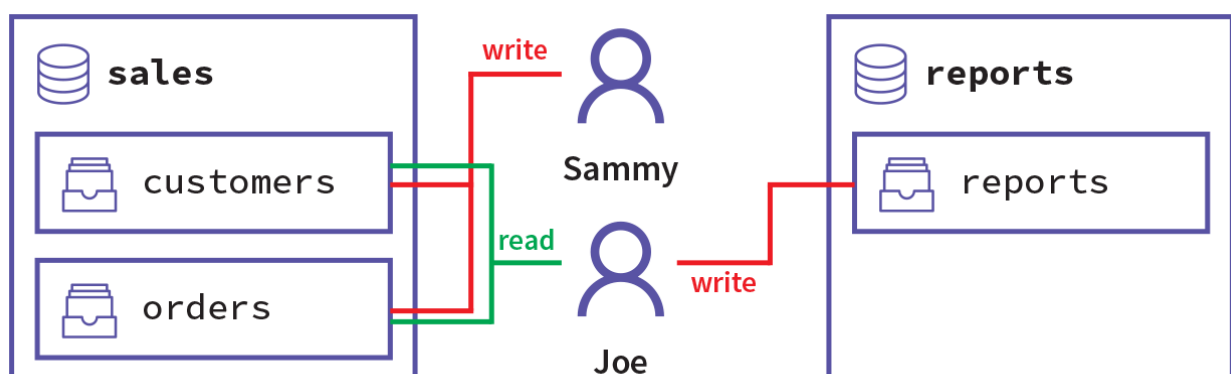
To explain how Role-Based Access Control — RBAC, for short — works in practice, this guide follows an example scenario with an imaginary sales company called Sammy Sales that uses two databases.

The first database (called **sales**) will store data about customer orders in the company shop with two separate collections: **customers** for their customers' personal data and **orders** for order details.

The second database (this one called **reports**) will store aggregated reports on monthly sales. This database will contain a single collection named **reports**.

The company has only two employees, both of whom have a level of database access that follows least privilege approach:

- Sammy, a sales representative, needs full access to both collections in the **sales** database, but has no need for working with the **reports** database.
- Joe, a sales analyst, needs write access to the **reports** database to construct reports as well as read-only access to the **sales** database to retrieve the data.



To create these sample databases, open the MongoDB shell on the server where you installed

```
$mongosh -u AdminSammy -p --authenticationDatabase admin
```

When prompted, enter the password that you set during installation to get access to the shell.

You can verify that you have access to the entire MongoDB instance by issuing the `show dbs` command:

```
> show dbs
```

This will return a list of all the databases currently available:

```
Output
admin    0.000GB
config   0.000GB
local    0.000GB
```

After confirming that you can access these databases, switch to the `sales` database:

```
> use sales
```

The shell will reply with a short confirmation

```
Output
switched to db sales
```

In MongoDB, there is no explicit action to create a database. A database is only created when it stores at least one document. With this in mind, you will need to insert some sample documents to prepare the databases and collections used in examples throughout this guide.

You are now in the `sales` database, but it won't actually exist until you insert something into it.

Create a collection named `customers` within `sales` and simultaneously insert a new document into it with the following operation:

```
db.customers.insertOne({name: 'Sammy'})
```

This example document only contains a `name` field with the value `'Sammy'`. Note that the data itself is not relevant for showcasing how the access rights work in practice, so this step outlines how to create database documents that only contain example mock data.

MongoDB will confirm the insertion with:

```
Output  
WriteResult({ "nInserted" : 1 })
```

Repeat this process, but this time create a collection named `orders`. To do this, run the following command. This time, the document's only field is `total` and it has a value of `100`:

```
db.orders.insertOne({total: 100})
```

MongoDB will once again confirm that the document was properly inserted.

Since you will be working with two databases, you will need to prepare the `reports` database as well. To do so, first switch to the `reports` database:

```
use reports
```

And insert another document, this time into `reports` collection:

```
db.reports.insertOne({orders: 1})
```

To confirm that both databases have been properly prepared, issue the `show dbs` command once again.

```
show dbs
```

After running this command a second time, the result will show two new entries for newly-created databases. These databases only persisted after you created the first documents within each of them:

```
Output
admin      0.000GB
config     0.000GB
local      0.000GB
reports    0.000GB
sales      0.000GB
```

The sample databases are now ready. Now, you can create a pair of users who will have the least amount of access privileges to the newly-created databases needed for this example scenario.

Step 2 — Creating the First User

In this step, you'll create the first of two MongoDB users. This first user will be for Sammy, the company's sales representative. This account will need full access to the `sales` database, but no access whatsoever to the `reports` database.

For this, we'll use the built-in `readWrite` role to grant both read and write access to the resources in the `sales` database. Since Sammy is a sales representative, we'll also use the `sales` database as the authentication database for the newly-created user.

First, switch to the `sales` database:

```
use sales
```

The shell will return a confirmation that you're using the chosen database:

```
Output
switched to db sales
```

Because Sammy works in the sales department, their MongoDB user account will be created with `sales` as the authentication database.

Run the following method to create the **sammy** user:

```
db.createUser(  
  {  
    user: "sammy",  
    pwd: passwordPrompt(),  
    roles: [  
      { role: "readWrite", db: "sales" }  
    ]  
  }  
)
```

This `createUser` method includes the following objects:

- `user` represents the username, which is **sammy** in this example.
- `pwd` represents the password. By using `passwordPrompt()` you will ensure that MongoDB shell will ask for the password when executing the command to be entered.
- `roles` is the list of roles granted. This example assigns **sammy** the `readWrite` role, granting them read and write access to the `sales` database. No other roles are assigned to **sammy** now, which means this user will not have any additional access rights immediately after creation.

```
...  
  pwd: "password",  
...
```

If this method is successful, it will return a confirmation message from the MongoDB shell similar to the following:

```
Output  
Successfully added user: {  
  "user" : "sammy",  
  "roles" : [  
    {  
      "role" : "readWrite",  
      "db" : "sales"  
    }  
  ]  
}
```



You can now verify that the new user can log in to the database and whether their access rights you specified are properly enforced.

You will keep the current MongoDB shell with your administrative user logged in open for later, so open **a separate server session**.

From the new server session, open the MongoDB shell. This time, specify specify **sammy** as the user and **sales** as the authentication database:

```
$mongosh -u sammy -p --authenticationDatabase sales
```

Enter the password you set when creating the **sammy** user. After accessing the shell prompt, execute the **show dbs** command to list the available databases:

```
show dbs
```

In contrast with your administrative account, only one database will be listed for **sammy**, as you've only granted them access to the **sales** database.

```
Output  
sales 0.000GB
```

Now check whether **sammy** can retrieve objects from both collections in the **sales** database. Switch to the **sales** database:

```
use sales
```

Then try to retrieve all customers:

```
db.customers.find()
```

This **find** command will return the document you created in this collection in Step 1:

```
Output
```

```
{ "_id" : ObjectId("60d888946ae8ac2c9120ec40"), "name" : "Sammy" }
```

You can also confirm that the second collection, `orders`, is available as intended:

```
db.orders.find()
```

Because you granted **sammy** the `readWrite` role, they are authorized to write new documents to this database. MongoDB will confirm the insertion was completed successfully:

Output

```
WriteResult({ "nInserted" : 1 })
```

Lastly, verify whether **sammy** can access the `reports` database. They will not be able to read or write any data in this database, as you did not grant them access through the assigned roles.

Switch to the `reports` database:

```
use reports
```

This `use` command will not result in any error on its own. Try accessing the document you inserted in Step 1 by running the following:

```
db.reports.find()
```

Now, MongoDB will throw an error message instead of returning any objects:

Output

```
Error: error: {
  "ok" : 0,
  "errmsg" : "not authorized on reports to execute
command { find: \"reports\", filter: {}, lsid: { id:
UUID(\"cca9e905-89f8-4903-ae12-46f23b43b967\") }, $db:
\"reports\" }",
  "code" : 13,
  "codeName" : "Unauthorized"
}
```

The `Unauthorized` error message tells you that **sammy** does not have enough access rights to interact with the data in the `reports` database.

So far, you've created the first database user with limited privileges and verified the access rights are properly enforced. Next, you will create a second user with different privileges.

Step 3 — Creating the Second User

Having created the **sammy** MongoDB user for Sammy, the sales representative, you still need an account for Joe, the company's sales analyst. Recall from the example scenario that Joe's job function requires a different set of privileges for the databases.

The process of creating this new user account is similar to the process you followed to create the **sammy** user.

Return to the server session where your administrative user is logged in to the MongoDB shell. From there, switch to `reports` database:

```
use reports
```

Because Joe works in the reporting department, their MongoDB user account will be created with `reports` as the authentication database.

Create the new **joe** user with the following command:

```
db.createUser(  
  {  
    user: "joe",  
    pwd: passwordPrompt(),  
    roles: [  
      { role: "readWrite", db: "reports" },  
      { role: "read", db: "sales" }  
    ]  
  }  
)
```

Notice the differences between the method used to create **joe** and the one used to create **sammy** in the previous step. This time, you assign two separate roles:

- `readWrite` applied to the `reports` database means **joe** will be able to read and write sales report data to this database
- `read` applied to the `sales` database makes sure that **joe** can access the sales data, but will not be able to write any documents into that database

Both of these roles are built-in MongoDB roles.

This command will return a confirmation message similar to the following:

```
Output
Successfully added user: {
  "user" : "joe",
  "roles" : [
    {
      "role" : "readWrite",
      "db" : "reports"
    },
    {
      "role" : "read",
      "db" : "sales"
    }
  ]
}
```

Next, verify that the new user's permissions are being properly enforced.

Once again, open **another server session**, as you'll make use of both the administrative MongoDB user and the **sammy** user in a later step.

Open the MongoDB shell, this time specifying **joe** as the user and `reports` as the authentication database:

```
$mongosh -u joe -p --authenticationDatabase reports
```

When prompted, enter the password you set when creating the **joe** user. Once you have access to the shell prompt, execute the `show dbs` command to list the databases available to **joe**:

```
show dbs
```

Since **joe** can use both the `sales` and `reports` databases, those two databases will be listed in the output:

Output

```
reports  0.000GB
sales    0.000GB
```

Now you can check whether **joe** can retrieve objects from the `sales` database.

Switch to `sales`:

```
use sales
```

Run the following `find` command to try to retrieve all orders:

```
db.orders.find()
```

Assuming you set up permissions correctly, this command will return the lone document you created in this collection in Step 1:

Output

```
{ "_id" : ObjectId("60d890730d31cc50dedea6ff"), "total" : 100 }
```

Next, try inserting a new document into the `orders` collection:

```
db.orders.insertOne({total: 50})
```

Because you assigned **joe** only a `read` role for this database, this `insert` command will fail with an error message:

Output

```
WriteCommandError({
  "ok" : 0,
  "errmsg" : "not authorized on sales to execute command
{ insert: \"orders\", ordered: true, lsid: { id:
```

```
UUID(\"ebbe853b-e269-463f-a1d4-2c5a5accb966\") }, $db:
\"sales\" }",
  "code" : 13,
  "codeName" : "Unauthorized"
})
```

The `Unauthorized` message tells you the reason behind the failure — the access rights `joe` has are not enough to insert a new document.

Next, confirm whether `joe` can read and write data in the `reports` database.

Switch to `reports`:

```
use reports
```

Then, try accessing the data within it using the `find` command:

```
db.reports.find()
```

Since `joe` is allowed to read from the database, MongoDB will respond with the list of currently available documents in this collection:

Output

```
{ "_id" : ObjectId("60d8897d6ae8ac2c9120ec41"),
  "orders" : 1 }
```

Then try inserting a new report by running the following command:

```
db.reports.insert({orders: 2})
```

This command will also succeed with the output message similar to this:

Output

```
WriteResult({ "nInserted" : 1 })
```

With that, you've created the second database user with limited privileges, but this time you granted them roles to two separate databases. You also verified that their access rights are properly enforced by the database server. Next, you will grant and then revoke additional permissions to one of the existing users.

Step 4 — Granting and Revoking Roles for Existing Users

In steps 2 and 3, you created new users and assigned them to roles during creation process. In practice, database administrators often need to revoke or grant new privileges to users that have already been created in the system. In this step, you will grant the **sammy** user a new role to allow them to access the `reports` database and revoke that permission shortly afterwards.

From **the administrative shell**, switch to the `sales` database where the user **sammy** was created:

```
use sales
```

To verify the user **sammy** is there, execute the `show users` command:

This command will return a list of all the users in this database as well as their respective roles:

```
Output
{
  "_id" : "sales.sammy",
  "userId" : UUID("cbc8ac18-37d8-4531-a52b-e7574044abcd"),
  "user" : "sammy",
  "db" : "sales",
  "roles" : [
    {
      "role" : "readWrite",
      "db" : "sales"
    }
  ],
  "mechanisms" : [
    "SCRAM-SHA-1",
    "SCRAM-SHA-256"
  ]
}
```

To assign a new role to this user, you can use the `grantRolesToUser` method. This method accepts the user's name and the list of roles to be granted in the same syntax as used when creating a new user.

The goal in this step is to grant **sammy** read-only permissions for `reports` database, so assign them the `read` role for that database:

```
db.grantRolesToUser("sammy", [{role: "read", db: "reports"}])
```

The command yields no output unless there was an error, so the lack of any message is an expected behavior. You can verify the command has taken effect by executing the `show users` command again:

`show users`

This command will return output similar to the following:

```
Output
{
  "_id" : "sales.sammy",
  "userId" : UUID("cbc8ac18-37d8-4531-a52b-e7574044abcd"),
  "user" : "sammy",
  "db" : "sales",
  "roles" : [
    {
      "role" : "read",
      "db" : "reports"
    },
    {
      "role" : "readWrite",
      "db" : "sales"
    }
  ],
  "mechanisms" : [
    "SCRAM-SHA-1",
    "SCRAM-SHA-256"
  ]
}
```

Notice the newly-added role in the `roles` section.

Now you can check whether **sammy** is indeed able to access the `reports` database after the change. Switch to the terminal window with **sammy** logged in and try accessing the reports once again.

Switch to the `reports` database:

```
use reports
```

And then run the `find` command on the `reports` collection:

```
db.reports.find()
```

Last time, the command failed with an error message. This time, though, it will return the list of documents in the `reports` database:

Output

```
{ "_id" : ObjectId("60d8897d6ae8ac2c9120ec41"),  
  "orders" : 1 }  
{ "_id" : ObjectId("60d899cafe3d26bf80e947fd"),  
  "orders" : 2 }
```

After some time you might want to revoke the **sammy** user's ability to access the reports. To illustrate this, go back to **the administrative console** and execute the following command, which will revoke the **sammy** user's `read` privileges on the `reports` database:

```
db.revokeRolesFromUser("sammy", [{role: "read", db: "reports"}])
```

The `revokeRolesFromUser` method takes the same set of arguments as `grantRolesToUser`, but instead removes the specified roles.

Once again, you can verify that the role is no longer available with `show users`:

Output

```
{  
  "_id" : "sales.sammy",
```

```

    "userId" : UUID("cbc8ac18-37d8-4531-a52b-
e7574044abcd"),
    "user" : "sammy",
    "db" : "sales",
    "roles" : [
      {
        "role" : "readWrite",
        "db" : "sales"
      }
    ],
    "mechanisms" : [
      "SCRAM-SHA-1",
      "SCRAM-SHA-256"
    ]
  }
}

```

To double check that **sammy** can no longer read from the **reports** database, try re-running the previous **find** command in the shell with **sammy** logged in:

```
db.reports.find()
```

This time the command will once again fail with an **Unauthorized** error:

Output

```

Error: error: {
  "ok" : 0,
  "errmsg" : "not authorized on reports to execute
command { find: \"reports\", filter: {}, lsid: { id:
UUID(\"2c86fba2-7615-40ae-9c3b-2dfdac2ed288\") }, $db:
\"reports\" }",
  "code" : 13,
  "codeName" : "Unauthorized"
}

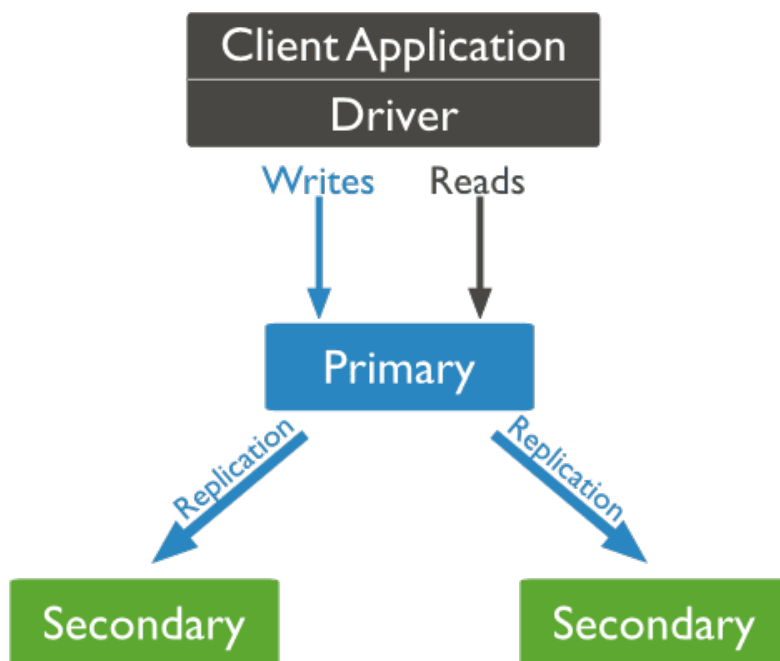
```

This shows that your revocation of the **sammy** user's **read** role was successful.

MongoDB Replica Set

Data replication is a must to keep your data protected, ensure high data availability, and ease of access, especially in the event of any unexpected error such as a system crash, hardware or software-based error, etc. Replication in MongoDB takes place with the help of MongoDB Replica Sets, each of which contains a set of MongoDB instances.

What is Replication in MongoDB?



It is the process of making data available across multiple data servers. Data replication ensures data security not only in the case of a single server failure but also in the case of a hardware failure. Replication stores multiple data copies across various data servers, allowing users to recover data whenever required.

MongoDB supports replication with the help of Replica Sets. These MongoDB Replica Sets are a combination of various mongod instances, each of them having a single primary node and multiple secondary nodes. The secondary node automatically copies the changes made to the primary, ensuring the same data is maintained across all servers. During the time of a failover or recovery process, the election mechanism chooses the new primary node. Replicating data from MongoDB can be a tiresome task without the right set of tools.

Key features of replication:

- **Scalability:** As the data volume increases the complexity of accessing data and working with data also increases. With replication in place, multiple data copies are available, allowing users to not only increase their data reserves but also recover any previous version in case of any errors or failures.
- **Performance:** When data is available across multiple machines and servers, it not only makes accessing data easier but also makes recovering from unexpected and sudden failures much easier. replication ensures data availability and security at all times.
- **Availability:** With replication in place, there's no need to worry about data failures. In situations where your primary source of data fails, you can easily access the same up-to-date data from a secondary reserve. This highly promotes data availability.

Why do you need Replication?

Replication is synchronizing data across multiple servers or storage space. It gives redundant data and allows users to store multiple copies of data in different servers to make it available all the time. Replication avoids data losses in events of uncertainty by making copies.

Replication is needed because of the following points listed below:

- It ensures data safety and keeps data safe on multiple servers.
- Replication of data means less traffic on one server and it makes data available all the time.
- Keeping copy data allow users to recover data from any of the secondary server if any disaster occurs to prevent any data loss.
- If there is any server with system failure or downtime for maintenance or indexing then another server can be used to deliver data.

How does Replication Work in MongoDB?

MongoDB uses replica sets to achieve replication. A MongoDB replica set is a collection of MongoDB instances that all store identical data. One node is the primary node in a replica, and it gets all write operations. All other instances, such as secondaries, use the primary's actions to ensure they have the same data set. There can only be one primary node in a replica set.

- A replica set is a collection of two or more nodes that are identical (generally a minimum of 3 nodes are required).
- One node in a replica set is the major node, while the others are secondary.
- From the primary to the secondary node, all data is replicated.
- Election for primary is established during automatic failover or maintenance, and a new primary node is elected.
- After a failed node is recovered, it rejoins the replica set and becomes a secondary node.

Replica Set Features

- N nodes in a cluster.
- Any node can be the primary.
- All write operations are sent to the primary storage.
- Failover occurs automatically.
- Recovery is done automatically.
- Primary election by consensus.

Example

```
$mongod --port 27017 --dbpath /usr/local/var/mongodb
--replSet rs1
```

- On port 27017, it will launch a mongod instance with the name **rs1**.
- Connect to this mongod instance using the command prompt.
(>test)
- To start a new replica set, use the Mongo client function **rs.initiate()**.
- Use the command **rs.conf()** to inspect the replica set setup (). Use the command **rs.status()** to verify the status of a replica set.

Initiating Replication in MongoDB

Once you've created directories for your Replica Set members, you can start the MongoDB instances for each of them using the following set of commands:

Use the following command for Server 1:

```
mongod --replSet rs0 --port 27017 --bind_ip
localhost,<hostname(s)|ip address(es)> --dbpath /srv/
mongodb/rs0-0 --oplogSize 128
```

Use the following command for Server 2:

```
mongod --replSet rs0 --port 27018 --bind_ip  
localhost,<hostname(s)|ip address(es)> --dbpath /srv/  
mongodb/rs0-0 --oplogSize 128
```

Use the following command for Server 3:

```
mongod --replSet rs0 --port 27019 --bind_ip  
localhost,<hostname(s)|ip address(es)> --dbpath /srv/  
mongodb/rs0-0 --oplogSize 128
```

The `--oplogSize` parameter is used to prevent the machine from getting overloaded during the test phase. It helps reduce the amount of disk space each disk consumes.

Connect to one of the various MongoDB instances that are a part of your ReplicaSet using the Mongo Shell. You can use the following command to connect using the port number:

```
mongo --port 27017
```

Use the **`rs.initiate()`** command to start the replication process and create a configuration object as follows by replacing the hostname parameter with your system name:

```
rs conf = {  
  _id: "rs0",  
  members: [  
    { _id: 0, host: "<hostname>:27017"},  
    { _id: 1, host: "<hostname>:27018"},  
    { _id: 2, host: "<hostname>:27019"}  
  ] }  
}
```

Pass the configuration object file as the parameter for the `initiate` command and use it as follows:

```
rs.initiate(rsconf)
```

This is how you can create a MongoDB Replica Set for testing and development purposes.

For further information on creating Replica Sets in MongoDB, you can check the [official documentation here](#).

Adding Members to Replica Set

Start mongod instances on various machines to add members to the replica set. Start a mongo client and use **rs.add()** command.

The **rs.add()** command has the following basic syntax:

```
>rs.add(HOST_NAME:PORT)
```

Example:

Assume **mongod1.net** is your mongod instance, and it's listening on port **27017**. Use the Mongo client command **rs.add()** to add this instance to the replica set.

```
>rs.add("mongod1.net:27017")
```

Only after you're connected to the primary node can you add a mongod instance to the replica set. Use the mongo client command **db.isMaster()** to see if you're connected to primary or not.

Schema Validation

Schema validation lets you create validation rules for your fields, such as allowed data types and value ranges.

MongoDB uses a flexible schema model, which means that documents in a collection do not need to have the same fields or data types by default.

When to Use Schema Validation

Schema validation is most useful for an established application where you have a good sense of how to organize your data. You can use schema validation in the following scenarios:

- For a users collection, ensure that the password field is only stored as a string. This validation prevents users from saving their password as an unexpected data type, like an image.
- For a sales collection, ensure that the item field belongs to a list of items that your store sells. This validation prevents a

user from accidentally misspelling an item name when entering sales data.

- For a students collection, ensure that the gpa field is always a positive number. This validation catches typos during data entry.

When MongoDB Checks Validation

When you create a new collection with schema validation, MongoDB checks validation during updates and inserts in that collection.

When you add validation to an existing, non-empty collection:

- Newly inserted documents are checked for validation.
- Documents already existing in your collection are not checked for validation until they are modified. Specific behavior for existing documents depends on your chosen validation level.

Tasks

For common tasks involving schema validation, see the following pages:

- [Specify JSON Schema Validation](#)
- [Specify Validation With Query Operators](#)
- [Specify Allowed Field Values](#)
- [View Existing Validation Rules](#)
- [Modify Schema Validation](#)
- [Query for and Modify Valid or Invalid Documents](#)
- [Bypass Schema Validation](#)

```
use admin
db.createUser(
```

```
{
  user: "myUserAdmin",
  pwd: passwordPrompt(), // or cleartext password
  roles: [
    { role: "userAdminAnyDatabase", db: "admin" },
    { role: "readWriteAnyDatabase", db: "admin" }
  ]
}
)
Password: mongodb
```

```
$mongo -u myUserAdmin -p --authenticationDatabase admin
```

This document contains the following information:

- **name**: the peak's name.
- **height**: the peak's elevation, in meters.
- **location**: the countries in which the mountain is located. This field stores values as an array to allow for mountains located in more than one country.
- **ascents**: this field's value is another document. When one document is stored within another document like this, it's known as an embedded or nested document. Each **ascents** document describes successful ascents of the given mountain. Specifically, each **ascents** document contains a **total** field that lists the total number of successful ascents of each given peak. Additionally, each of these nested documents contain two fields whose values are also nested documents:
 - **first**: this field's value is a nested document that contains one field, **year**, which describes the year of the first overall successful ascent.
 - **first_winter**: this field's value is a nested document that also contains a **year** field, the value of which represents the year of the first successful winter ascent of the given mountain.

```
db.peaks.insertOne(
  {
    "name": "Everest",
    "height": 8848,
```

```

    "location": ["Nepal", "China"],
    "ascents": {
      "first": {
        "year": 1953
      },
      "first_winter": {
        "year": 1980
      },
      "total": 5656
    }
  }
)

```

The output will contain a success message and an object identifier assigned to the newly inserted object:

```

Output
{
  "acknowledged" : true,
  "insertedId" : ObjectId("618ffa70bfa69c93a8980443")
}

```

For a database containing a list of the highest mountains in the world, adding a document representing a mountain but not including its name would likely be a serious error.

```

{
  "name": "Manaslu",
  "height": "8163m",
  "location": "Nepal"
}
db.peaks.insertMany([
  {
    "height": 8611,
    "location": ["Pakistan", "China"],
    "ascents": {
      "first": {
        "year": 1954
      },
      "first_winter": {

```

```

        "year": 1921
      },
      "total": 306
    }
  },
  {
    "name": "Manaslu",
    "height": "8163m",
    "location": "Nepal"
  }
])

```

As it turns out, MongoDB will not return any errors and both documents will be inserted successfully:

```

Output
{
  "acknowledged" : true,
  "insertedIds" : [
    ObjectId("618ffd0bbfa69c93a8980444"),
    ObjectId("618ffd0bbfa69c93a8980445")
  ]
}

```

As this output indicates, both of these documents are valid JSON, which is enough to insert them into the collection.

Any given collection can only use a single JSON Schema, but you can assign a schema when you create the collection or any time afterwards. If you decide to change your original validation rules later on, you will have to replace the original JSON Schema document with one that aligns with your new requirements.

To assign a JSON Schema validator document to the `peaks` collection you created in the previous step, you could run the following command:

```

db.runCommand({
  "collMod": "collection_name",
  "validator": {
    $jsonSchema: {JSON_Schema_document}
  }
})

```

```
}  
})
```

The `runCommand` method executes the `collMod` command, which modifies the specified collection by applying the `validator` attribute to it. The `validator` attribute is responsible for schema validation and, in this example syntax, it accepts the `$jsonSchema` operator.

use admin

Then run a `grantRolesToUser()` method and grant your user the `dbAdmin` role over the database where you created the `peaks` collection. The following example assumes the `peaks` collection is in the `test` database:

```
db.grantRolesToUser(  
  "myUserAdmin",  
  [ { role : "dbAdmin", db : "test" } ]  
)
```

Alternatively, you can grant your user the `dbAdminAnyDatabase` role. As this role's name implies, it will grant your user `dbAdmin` privileges over every database on your MongoDB instance:

```
db.grantRolesToUser(  
  "AdminSammy",  
  [ "dbAdminAnyDatabase" ]  
)
```

After granting your user the appropriate role, navigate back to the database where your `peaks` collection is stored:

```
>use test
```

Be aware that you can also assign a JSON Schema validator when you create a collection. To do so, you could use the following syntax:

```
db.createCollection(  
  "collection_name", {  
    "validator": {
```

```

    $jsonSchema: {JSON_Schema_document}
  }
})

```

The JSON schema document you pass to the `validator` attribute should outline every validation rule you want to apply to the collection. The following example JSON Schema will make sure that the `name` field is present in every document in the collection, and that the `name` field's value is always a string:

```

{
  "bsonType": "object",
  "description": "Document describing a mountain peak",
  "required": ["name"],
  "properties": {
    "name": {
      "bsonType": "string",
      "description": "Name must be a string and is required"
    }
  },
}

```

This schema document outlines certain requirements that certain parts of documents entered into the collection must follow. The root part of the JSON Schema document (the fields before `properties`, which in this case are `bsonType`, `description`, and `required`) describes the database document itself.

```

db.runCommand({
  "collMod": "peaks",
  "validator": {
    $jsonSchema: {
      "bsonType": "object",
      "description": "Document describing a mountain peak",
      "required": ["name"],
      "properties": {
        "name": {
          "bsonType": "string",
          "description": "Name must be a string and is required"
        }
      }
    }
  },
})

```

```
}  
}  
})
```

MongoDB will respond with a success message indicating that the collection was successfully modified:

Output

```
{ "ok" : 1 }
```

```
db.peaks.insertOne(  
  {  
    "height": 8611,  
    "location": ["Pakistan", "China"],  
    "ascents": {  
      "first": {  
        "year": 1954  
      },  
      "first_winter": {  
        "year": 1921  
      },  
      "total": 306  
    }  
  }  
)
```

This time, the operation will trigger an error message indicating a failed document validation:

Output

```
WriteError({  
  "index" : 0,  
  "code" : 121,  
  "errmsg" : "Document failed validation",  
  ...  
})
```

You can also test whether MongoDB will enforce the data type requirement you included in the JSON Schema by running the following `insertOne()` method. This is similar to the last operation, but this time it includes a `name` field. However, this field's value is a number instead of a string:

```

db.peaks.insertOne(
  {
    "name": 123,
    "height": 8611,
    "location": ["Pakistan", "China"],
    "ascents": {
      "first": {
        "year": 1954
      },
      "first_winter": {
        "year": 1921
      },
      "total": 306
    }
  }
)

```

Once again, the validation will fail. Even though the `name` field is present, it doesn't meet the constraint that requires it to be a string:

```

Output
WriteError({
  "index" : 0,
  "code" : 121,
  "errmsg" : "Document failed validation",
  ...
})

```

```

db.peaks.insertOne(
  {
    "name": "K2"
  }
)

```

The operation will succeed, and the document will receive the object identifier as usual:

```

Output
{
  "acknowledged" : true,
  "insertedId" : ObjectId("61900965bfa69c93a8980447")
}

```

```
}
```

The schema validation rules pertain only to the `name` field. At this point, as long as the `name` field fulfills the validation requirements, the document will be inserted without error. The rest of the document can take any shape.

With that, you've created your first JSON Schema document and applied the first schema validation rule to the `name` field, requiring it to be present and a string. However, there are different validation options for different data types. Next, you'll validate number values stored in each document's `height` field.

Step 3 — Validating Number Fields

Recall from Step 1 when you inserted the following document into the `peaks` collection:

```
{
  "name": "Manaslu",
  "height": "8163m",
  "location": "Nepal"
}
```

```
db.runCommand({
  "collMod": "peaks",
  "validator": {
    $jsonSchema: {
      "bsonType": "object",
      "description": "Document describing a mountain peak",
      "required": ["name", "height"],
      "properties": {
        "name": {
          "bsonType": "string",
          "description": "Name must be a string and is required"
        },
        "height": {
          "bsonType": "number",
```

```

        "description": "Height must be a number and is
required"
    },
    {
        "height": 8300
    }
}
})

```

In this command's schema document, the `height` field is included in the `required` array. Likewise, there's a `height` document within the `properties` object that will require any new `height` values to be a number. Again, the `description` field is auxiliary, and any description you include should only be to help other users understand the intention behind the JSON Schema.

MongoDB will respond with a short success message to let you know that the collection was successfully modified:

```

Output
{ "ok" : 1 }

```

```

db.peaks.insertOne(
  {
    "name": "Test peak",
    "height": 8300
  }
)

```

The insertion will succeed:

```

Output
{
  acknowledged: true,
  insertedId: ObjectId("61e0c8c376b24e08f998e371")
}

```

Next, try inserting a document with a missing `height` field:

```

db.peaks.insertOne(
  {
    "name": "Test peak"
  }
)

```

```
)  
}
```

Then try another that includes the `height` field, but this field contains a string value:

```
db.peaks.insertOne(  
  {  
    "name": "Test peak",  
    "height": "8300m"  
  }  
)
```

Both times, the operations will trigger an error message and fail:

```
Output  
WriteError({  
  "index" : 0,  
  "code" : 121,  
  "errmsg" : "Document failed validation",  
  ...  
})
```

However, if you try inserting a mountain peak with a negative height, the mountain will save properly:

```
db.peaks.insertOne(  
  {  
    "name": "Test peak",  
    "height": -100  
  }  
)
```

To prevent this, you could add a few more properties to the schema validation document. Replace the current schema validation settings by running the following operation:

```
db.runCommand({  
  "collMod": "peaks",
```

```

"validator": {
  $jsonSchema: {
    "bsonType": "object",
    "description": "Document describing a mountain peak",
    "required": ["name", "height"],
    "properties": {
      "name": {
        "bsonType": "string",
        "description": "Name must be a string and is required"
      },
      "height": {
        "bsonType": "number",
        "description": "Height must be a number between 100
and 10000 and is required",
        "minimum": 100,
        "maximum": 10000
      }
    }
  }
}
})

```

The new `minimum` and `maximum` attributes set constraints on values included in `height` fields, ensuring they can't be lower than 100 or higher than 10000. This range makes sense in this case, as this collection is used to store information about mountain peak heights, but you could choose any values you like for these attributes.

Now, if you try inserting a peak with a negative `height` value again, the operation will fail:

```

db.peaks.insertOne(
  {
    "name": "Test peak",
    "height": -100
  }
)

```

Output

```
WriteError({
  "index" : 0,
  "code" : 121,
  "errmsg" : "Document failed validation",
  ...
})
```

As this output shows, your document schema now validates string values held in each document's `name` field as well as numeric values held in the `height` fields. Continue reading to learn how to validate array values stored in each document's `location` field.

Step 4 — Validating Array Fields

Now that each peak's `name` and `height` values are being verified by schema validation constraints, we can turn our attention to the `location` field to guarantee its data consistency.

Specifying the location for mountains is more tricky than one might expect, since peaks span more than one country, and this is the case for many of the famous eight-thousanders. Because of this, it would make sense store each peak's `location` data as an array containing one or more country names instead of being just a string value. As with the `height` values, making sure each `location` field's data type is consistent across every document can help with summarizing data when using aggregation pipelines.

First, consider some examples of `location` values that users might enter, and weigh which ones would be valid or invalid:

- `["Nepal", "China"]`: this is a two-element array, and would be a valid value for a mountain spanning two countries.
- `["Nepal"]`: this example is a single-element array, it would also be a valid value for a mountain located in a single country.
- `"Nepal"`: this example is a plain string. It would be invalid because although it lists a single country, the `location` field should always contain an array

- `[]`: an empty array, this example would not be a valid value. After all, every mountain must exist in at least one country.
- `["Nepal", "Nepal"]`: this two-element array would also be invalid, as it contains the same value appearing twice.
- `["Nepal", 15]`: lastly, this two-element array would be invalid, as one of its values is a number instead of a string and this is not a correct location name.

```
db.runCommand({
  "collMod": "peaks",
  "validator": {
    $jsonSchema: {
      "bsonType": "object",
      "description": "Document describing a mountain peak",
      "required": ["name", "height", "location"],
      "properties": {
        "name": {
          "bsonType": "string",
          "description": "Name must be a string and is required"
        },
        "height": {
          "bsonType": "number",
          "description": "Height must be a number between 100
and 10000 and is required",
          "minimum": 100,
          "maximum": 10000
        },
        "location": {
          "bsonType": "array",
          "description": "Location must be an array of strings",
          "minItems": 1,
          "uniqueItems": true,
          "items": {
            "bsonType": "string"
          }
        }
      }
    }
  },
}
```

```
}  
})
```

In this `$jsonSchema` object, the `location` field is included within the `required` array as well as the `properties` object. There, it's defined with a `bsonType` of `array` to ensure that the `location` value is always an array rather than a single string or a number.

Now try inserting documents matching the examples prepared earlier to test how the new rule behaves. Once again, let's use the minimal document structure, with only the `name`, `height`, and `location` fields present.

```
db.peaks.insertOne(  
  {  
    "name": "Test peak",  
    "height": 8300,  
    "location": ["Nepal", "China"]  
  }  
)
```

The document will be inserted successfully as it fulfills all the defined validation expectations. Similarly, the following document will insert without error:

```
db.peaks.insertOne(  
  {  
    "name": "Test peak",  
    "height": 8300,  
    "location": ["Nepal"]  
  }  
)
```

However, if you were to run any of the following `insertOne()` methods, they would trigger a validation error and fail:

```
db.peaks.insertOne(  
  {
```

```

        "name": "Test peak",
        "height": 8300,
        "location": "Nepal"
    }
)

db.peaks.insertOne(
    {
        "name": "Test peak",
        "height": 8300,
        "location": []
    }
)

db.peaks.insertOne(
    {
        "name": "Test peak",
        "height": 8300,
        "location": ["Nepal", "Nepal"]
    }
)

db.peaks.insertOne(
    {
        "name": "Test peak",
        "height": 8300,
        "location": ["Nepal", 15]
    }
)

```

As per the validation rules you defined previously, the `location` values provided in these operations are considered invalid.

Step 5 — Validating Embedded Documents

At this point, your `peaks` collection has three fields — `name`, `height` and `location` — that are being kept in check by schema validation.

This step focuses on defining validation rules for the `ascents` field, which describes successful attempts at summiting each peak.

In the example document from Step 1 that represents Mount Everest, the `ascents` field was structured as follows:

You can change the schema validation document so that the `ascents` field must always be present and its value must always be a subdocument. This subdocument, in turn, must always contain a `total` attribute holding a number greater than or equal to zero. The `first` and `first_winter` fields aren't required for the purposes of this guide, so the validation form won't consider them and they can take flexible forms.

```
db.runCommand({
  "collMod": "peaks",
  "validator": {
    $jsonSchema: {
      "bsonType": "object",
      "description": "Document describing a mountain peak",
      "required": ["name", "height", "location", "ascents"],
      "properties": {
        "name": {
          "bsonType": "string",
          "description": "Name must be a string and is required"
        },
        "height": {
          "bsonType": "number",
          "description": "Height must be a number between 100
and 10000 and is required",
          "minimum": 100,
          "maximum": 10000
        },
        "location": {
          "bsonType": "array",
          "description": "Location must be an array of strings",
          "minItems": 1,
          "uniqueItems": true,
```

```

        "items": {
            "bsonType": "string"
        }
    },
    "ascents": {
        "bsonType": "object",
        "description": "Ascent attempts information",
        "required": ["total"],
        "properties": {
            "total": {
                "bsonType": "number",
                "description": "Total number of ascents must be 0
or higher",
                "minimum": 0
            }
        }
    }
},
}
})

```

Again, because neither the `first` nor the `first_winter` fields are mandatory for the purposes of this guide, they aren't included in these validation rules.

```

db.peaks.insertOne(
{
    "name": "Everest",
    "height": 8848,
    "location": ["Nepal", "China"],
    "ascents": {
        "first": {
            "year": 1953,
        },
        "first_winter": {
            "year": 1980,
        },
    },
}
)

```

```

        "total": 5656,
      }
    }
  )

```

The document saves successfully, and MongoDB returns the new object identifier:

Output

```

{
  "acknowledged" : true,
  "insertedId" : ObjectId("619100f51292cb2faee531f8")
}

```

To make sure the last pieces of validation work properly, try inserting a document that doesn't include the `ascents` field:

```

db.peaks.insertOne(
  {
    "name": "Everest",
    "height": 8848,
    "location": ["Nepal", "China"]
  }
)

```

This time, the operation will trigger an error message pointing out a failed document validation:

Output

```

WriteError({
  "index" : 0,
  "code" : 121,
  "errmsg" : "Document failed validation",
  ...
})

```

```

db.peaks.insertOne(
  {
    "name": "Everest",
    "height": 8848,
    "location": ["Nepal", "China"],
    "ascents": {

```

```

        "first": {
            "year": 1953,
        },
        "first_winter": {
            "year": 1980,
        }
    }
}
)

```

This will again trigger an error.

As a final test, try entering a document that contains an `ascents` field with a `total` value, but this value is negative:

```

db.peaks.insertOne(
{
    "name": "Everest",
    "height": 8848,
    "location": ["Nepal", "China"],
    "ascents": {
        "first": {
            "year": 1953,
        },
        "first_winter": {
            "year": 1980,
        },
        "total": -100
    }
}
)

```

Because of the negative `total` value, this document will also fail the validation test.

mongodb collection validation schema

- The fields: `username`, `email`, and `password` should be present in each document (they are mandatory).
- The fields: `username`, `email`, and `password` should be of type string, and their strings' length should be between the minimum and maximum limits.
- The field `email` should comply with a specific regex pattern.
- The field: `registrationdate` should be of type date.
- The fields: `confirmed` and `canceled` should be of type bool (Boolean: true or false).
- The fields `typeid` and `countryid` should be of type int (integer), and their values should be between a minimum and a maximum number.

{	
\$jsonSchema: {	
bsonType: 'object',	
additionalProperties: false,	
required: [	
username',	
email',	
password'	
],	
properties: {	
_id: {},	
username: {	
bsonType: 'string',	
minLength: 6,	
maxLength: 20,	
description: 'It is required and it must be a string with length between 6 and 20'	
},	
password: {	

	bsonType: 'string',
	maxLength: 80,
	description: 'It must be a string with max length 80'
	},
	email: {
	bsonType: 'string',
	minLength: 6,
	maxLength: 40,
	pattern: '[a-z0-9._%+!\$&*~^!~#%{}-/]+@([a-z0-9-]+.){1,}([a-z]{2,22})',
	description: 'It is required and it must be a string with length between 6 and 40
	},
	registrationdate: {
	bsonType: 'date',
	description: 'It must be a date'
	},
	confirmed: {
	bsonType: 'bool',
	description: 'It can only be true or false'
	},
	canceled: {
	bsonType: 'bool',
	description: 'It can only be true or false'
	},
	typeid: {
	bsonType: 'int',
	minimum: 1,
	maximum: 4,
	description: 'It must be an integer in [1, 5]'
	},
	countryid: {
	bsonType: 'int',
	minimum: 1,
	maximum: 250,
	description: 'It must be an integer in [1, 250]'
	}
	}

	}
	}

test>	use ticket-management
	switched to db ticket-management
	ticket-management>
	ticket-management> db.getCollectionInfos({name: "users"})
	[
	{
	name: 'users',
	type: 'collection',
	options: {
	validator: {
	'\$jsonSchema': {
	bsonType: 'object',
	additionalProperties: false,
	required: ['username', 'email', 'password'],
	properties: {
	_id: {},
	username: {
	bsonType: 'string',
	minLength: 6,
	maxLength: 20,
	description: 'It is required and it must be a string with length between 6 and 20'
	},
	password: {
	bsonType: 'string',
	maxLength: 80,
	description: 'It must be a string with max length 80'
	},
	email: {
	bsonType: 'string',
	minLength: 6,
	maxLength: 40,
	pattern: '[a-z0-9._%+!\$&*^~#%{}-]+@[a-z0-9-]+.{1,}([a-z]{2,22})',
	description: 'It is required and it must be a string with length between 6 and 40'

	,
	registrationdate: { bsonType: 'date', description: 'It must be a date' },
	confirmed: {
	bsonType: 'bool',
	description: 'It can only be true or false'
	},
	canceled: {
	bsonType: 'bool',
	description: 'It can only be true or false'
	},
	typeid: {
	bsonType: 'int',
	minimum: 1,
	maximum: 4,
	description: 'It must be an integer in [1, 5]'
	},
	countryid: {
	bsonType: 'int',
	minimum: 1,
	maximum: 250,
	description: 'It must be an integer in [1, 250]'
	}
	}
	}
	},
	validationLevel: 'strict',
	validationAction: 'error'
	},
	info: {
	readOnly: false,
	uuid: UUID("b033d158-5ee1-4187-9e29-70be69ed97bb")
	},
	idIndex: { v: 2, key: { _id: 1 }, name: '_id_' }
	}
	]
	ticket-management>

Specify Validation With Query Operators

You can specify validation using query operators such as `$eq` and `$gt` to compare fields.

Restrictions

- You can't specify the following [query operators](#) in a validator object:
 - `$expr` with `$function` expressions
 - [\\$near](#)
 - [\\$nearSphere](#)
 - [\\$text](#)
 - [\\$where](#)
- You can't specify schema validation for:
 - Collections in the `admin`, `local`, and `config` databases
 - [System collections](#)

Context

Consider an application that tracks customer orders. The orders have a base price and a VAT. The orders collection contains these fields to track total price:

- `price`
- `VAT`
- `totalWithVAT`

Steps

The following procedure creates a schema validation with query operators to ensure that totalWithVAT matches the expected combination of price and VAT.

Create a collection with validation.

Create an orders collection with schema validation:

```
db.createCollection( "orders",
{
  validator: {
    $expr:
    {
      $eq: [
        "$totalWithVAT",
        { $multiply: [ "$total", { $sum: [ 1, "$VAT" ] } ] }
      ]
    }
  }
})
```

With this validation, you can only insert documents if the totalWithVAT field equals $\text{total} * (1 + \text{VAT})$

Confirm that the validation prevents invalid documents.

The following operation fails because the totalWithVAT field does not equal the correct value:

```
db.orders.insertOne( {
  total: NumberDecimal("141"),
  VAT: NumberDecimal("0.20"),
  totalWithVAT: NumberDecimal("169")
})
```

$141 * (1 + 0.20)$ equals 169.2, so the value of the totalWithVAT field must be 169.2.

The operation returns this error:

MongoServerError: Document failed validation

Additional information: {
failingDocumentId: ObjectId("62bcc9b073c105dde9231293"),
details: {
operatorName: '\$expr',
specifiedAs: {
'\$expr': {
'\$eq': [
'\$totalWithVAT',
{
'\$multiply': ['\$total', { '\$sum': [1, '\$VAT'] }]
}
]
}
},
reason: 'expression did not match',
expressionResult: false
}
}

Make the document valid and insert it.

After updating the document to have the correct totalWithVAT value, the operation succeeds:

```
db.orders.insertOne( {
  total: NumberDecimal("141"),
  VAT: NumberDecimal("0.20"),
  totalWithVAT: NumberDecimal("169.2")
})
```

MongoDB returns the following output, indicating that the insert was successful:

{
acknowledged: true,
insertedId: ObjectId("6304f4651e52f124b84479ba")

```
}
```

Additional Information

You can combine query operator validation with [JSON Schema validation](#).

For example, consider a sales collection with this schema validation:

```
db.createCollection( "sales", {  
  validator: {  
    "$and": [  
      // Validation with query operators  
      {  
        "$expr": {  
          "$lt": ["$lineItems.discountedPrice", "$lineItems.price"]  
        }  
      },  
      // Validation with JSON Schema  
      {  
        "$jsonSchema": {  
          "properties": {  
            "items": { "bsonType": "array" }  
          }  
        }  
      }  
    ]  
  }  
})
```

View Existing Validation Rules

You can view a collection's validation rules to determine what restrictions are imposed on documents and how MongoDB handles invalid documents when they occur. To view a collection's validation rules, use the [db.getCollectionInfos\(\)](#) method or [listCollections](#) database command.

Both commands return the same information, but the output format differs between each command.

Example: db.getCollectionInfos() Syntax

The following command uses `db.getCollectionInfos()` to return the validation rules for the students collection:

```
db.getCollectionInfos( { name: "students" } )[0].options.validator
```

The output resembles the following validation object:

{
'\$jsonSchema': {
bsonType: 'object',
required: ['name', 'year', 'major', 'address'],
properties: {
name: {
bsonType: 'string',
description: 'must be a string and is required'
},
year: {
bsonType: 'int',
minimum: 2017,
maximum: 3017,
description: 'must be an integer in [2017, 3017] and is required'
},
gpa: {
bsonType: ['double'],
description: 'must be a double if the field exists'
}
}
}

Example: listCollections Syntax

The following command uses `listCollections` to return the validation rules for the students collection:

```
db.runCommand ( { listCollections: 1, filter: { name: "students" } } )
```

The output resembles the following object:

{
cursor: {
id: Long("0"),
ns: 'test.\$cmd.listCollections',
firstBatch: [
{
name: 'students',
type: 'collection',
options: {
validator: {
'\$jsonSchema': {
bsonType: 'object',
required: ['name', 'year', 'major', 'address'],
properties: {
name: {
bsonType: 'string',
description: 'must be a string and is required'
},
gpa: {
bsonType: ['double'],
description: 'must be a double if the field exists'
}
}
},
validationAction: 'warn'
}
},
info: {
readOnly: false,

uuid: UUID ("bf560865-5879-4ec1-b389-f77a03abbc5a")
},
idIndex: { v: 2, key: { _id: 1 }, name: '_id_' }
}
]
},
ok: 1
}

Modify Schema Validation

After you add schema validation to a collection, you can modify your schema validation at any time. For example, you may decide:

- That documents in a `users` collection no longer require an email address.
- To increase the minimum length for a password field from 8 characters to 12.

To modify a collection's schema validation, use the [collMod](#) command and specify the updated validation in the validator object.

Context

You can modify all components of a schema validation, including its rules, validation level, and validation action.

If you update a collection's validation rules, documents inserted prior to the validation change may no longer be valid. How MongoDB handles these invalid documents depends on your `validationLevel`. By default, MongoDB applies validation checks to all documents, regardless of when they were inserted.

Steps

The following procedure creates a collection with validation rules and then modifies those rules. You will observe the results when inserting an invalid and valid document.

Create a collection with validation.

Create a `users` collection with validation rules:

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: [ "username", "password" ],
      properties: {
        name: {
          bsonType: "string",
          description: "must be a string and is required"
        },
        password: {
          bsonType: "string",
          minLength: 8,
          description: "must be a string of at least 8 characters, and
is required"
        }
      }
    }
  }
})
```

Modify the validation schema.

Run the following `collMod` command to change the `minLength` of the password field from 8 to 12:

```
db.runCommand( { collMod: "users",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: [ "username", "password" ],
```

```

    properties: {
      name: {
        bsonType: "string",
        description: "must be a string and is required"
      },
      password: {
        bsonType: "string",
        minLength: 12,
        description: "must be a string of at least 12 characters,
and is required"
      }
    }
  }
}
})

```

Results

The following examples show what happens when you insert a valid and invalid document into the users collection, and how to handle previously valid documents that are no longer valid as a result of changes to the validation rules.

Insert an Invalid Document

The following operation attempts to insert an invalid document. The document is invalid because the password field is 10 characters long when the minimum length is 12:

```

db.users.insertOne(
  {
    "username": "salesAdmin01",
    "password": "kT9$j4wg#M"
  }
)

```

MongoDB returns the following error:

MongoServerError: Document failed validation
Additional information : {
failingDocumentId : ObjectId("62be0adb73c105dde9231299"),
details : {
operatorName : '\$jsonSchema',
schemaRulesNotSatisfied : [
{
operatorName : 'properties',
propertiesNotSatisfied : [
{
propertyName : 'password',
description : 'must be a string of at least 8 characters, and is
details : [
{
operatorName : 'minLength',
specifiedAs : { minLength : 12 },
reason : 'specified string length was not satisfied',
consideredValue : 'kT9\$j4wg#M'
}
]
}
]
}
]
}
}

Handle a Previously Valid Document That Is No Longer Valid

Consider the following document that is valid for the first version of the schema validation, but not the second:

```
db.users.insertOne(
  {
    "username": "salesAdmin02",
    "password": "i8U60*VyL8"
```

```
}  
)
```

The document's password field is 10 characters. The first version of the schema validation required a minimum of 8 characters, meaning this document was valid. However, after updating the validation to require the password to be a minimum of 12 characters, the document is no longer valid.

Specify Validation Level for Existing Documents

For documents that already exist in your collection prior to adding validation, you can specify how MongoDB applies validation rules to these documents.

Context

Your schema's `validationLevel` determines the documents for which MongoDB applies validation rules:

Validation Level	Behavior
strict	(Default) MongoDB applies validation rules to all inserts and updates.
moderate	MongoDB only applies validation rules to existing valid documents. Updates to invalid documents which exist prior to the validation being added are not checked for validity.

The examples on this page use a `contacts` collection with these documents

```
db.contacts.insertMany([
```

```
{ "_id": 1, "name": "Anne", "phone": "+1 555 123 456", "city":  
"London", "status": "Complete" },  
{ "_id": 2, "name": "Ivan", "city": "Vancouver" }  
])
```

Steps: Use strict Validation

The following example adds a strict validation to the contacts collection and shows the results when attempting to update invalid documents

Specify validation rules with **strict** validation level.

Add a validator to the contacts collection with strict validationLevel:

```
db.runCommand( {  
  collMod: "contacts",  
  validator: { $jsonSchema: {  
    bsonType: "object",  
    required: [ "phone", "name" ],  
    properties: {  
      phone: {  
        bsonType: "string",  
        description: "phone must be a string and is required"  
      },  
      name: {  
        bsonType: "string",  
        description: "name must be a string and is required"  
      }  
    }  
  }  
},  
  validationLevel: "strict"  
})
```

Because the validationLevel is strict, when any document is updated, MongoDB checks that document for validation.

Test the validation.

The following update commands modify both documents in the contacts collection such that neither of the documents are consistent with the validation rule which requires name to be a string:

```
db.contacts.updateOne(
  { _id: 1 },
  { $set: { name: 10 } }
)
```

```
db.contacts.updateOne(
  { _id: 2 },
  { $set: { name: 20 } }
)
```

Observe results.

Both update operations fail. MongoDB returns the following output for each operation:

MongoServerError: Document failed validation
Additional information : {
failingDocumentId : <id>,
details : {
operatorName : '\$jsonSchema',
schemaRulesNotSatisfied : [
{
operatorName : 'properties',
propertiesNotSatisfied : [
{
propertyName : 'name',
description : 'name must be a string and is required',
details : [
{
operatorName : 'bsonType',
specifiedAs : { bsonType : 'string' },

reason: 'type did not match',
consideredValue: <value>,
consideredType: 'int'
}
]
}
]
},
{
operatorName: 'required',
specifiedAs: { required: ['phone', 'name'] },
missingProperties: ['phone']
}
]
}
}

Steps: Use moderate Validation

The following example adds a moderate validation to the contacts collection and shows the results when attempting to update invalid documents.

Specify validation rules with **moderate** validation level.

Add a validator to the contacts collection with moderate validationLevel:

```
db.runCommand( {
  collMod: "contacts",
  validator: { $jsonSchema: {
    bsonType: "object",
    required: [ "phone", "name" ],
    properties: {
      phone: {
        bsonType: "string",
        description: "phone must be a string and is required"
      },
    },
  },
}
```

```

    name: {
      bsonType: "string",
      description: "name must be a string and is required"
    }
  },
  validationLevel: "moderate"
})

```

Because the validationLevel is moderate:

- If you update the document with `_id: 1`, MongoDB applies the new validation rules because the existing document meets the validation requirements.
- If you update the document with `_id: 2`, MongoDB does not apply the new validation rules because the existing document does not meet the validation requirements.

Test the validation.

The following update commands modify both documents in the contacts collection such that neither of the documents are consistent with the validation rule which requires name to be a string:

```

db.contacts.updateOne(
  { _id: 1 },
  { $set: { name: 10 } }
)

```

```

db.contacts.updateOne(
  { _id: 2 },
  { $set: { name: 20 } }
)

```

Observe results.

MongoDB returns the following output for each operation:

```
// _id: 1
```

```
MongoServerError: Document failed validation
```

```
Additional information: {
```

```
  failingDocumentId: 1,
```

```
  details: {
```

```
    operatorName: '$jsonSchema',
```

```
    schemaRulesNotSatisfied: [
```

```
    {
```

```
      operatorName: 'properties',
```

```
      propertiesNotSatisfied: [
```

```
      {
```

```
        propertyName: 'name',
```

```
        description: 'name must be a string and is required',
```

```
        details: [
```

```
        {
```

```
          operatorName: 'bsonType',
```

```
          specifiedAs: { bsonType: 'string' },
```

```
          reason: 'type did not match',
```

```
          consideredValue: 10,
```

```
          consideredType: 'int'
```

```
        }
```

```
      ]
```

```
    }
```

```
  ]
```

```
}
```

```
]
```

```
}
```

```
}
```

```
// _id: 2
```

```
{
```

```
  acknowledged: true,
```

```
  insertedId: null,
```

```
  matchedCount: 1,
```

<code>modifiedCount: 0,</code>
<code>upsertedCount: 0</code>
<code>}</code>

The output shows that:

- The update fails for the document with `_id: 1`. This document met the initial validation requirements, and MongoDB applies validation rules to this document.
- The update succeeds for the document with `_id: 2`. This document did not meet the initial validation requirements, and MongoDB does not apply validation rules to this document.

Choose How to Handle Invalid Documents

You can specify how MongoDB handles documents that violate validation rules. When an operation would result in an invalid document, MongoDB can either:

- Reject any insert or update that violates the validation criteria. This is the default behavior.
- Allow the operation to proceed, but record the violation in the MongoDB log.

Rejecting invalid documents ensures that your schema stays consistent. However, in certain scenarios you may want to allow invalid documents, such as a data migration containing documents from before a schema was established.

Context

Your schema's `validationAction` option determines how MongoDB handles invalid documents:

Validation Action	Behavior
error	(Default) MongoDB rejects any insert or update that violates the validation criteria
warn	MongoDB allows the operation to proceed, but records the violation in the MongoDB log.

Option 1: Reject Invalid Documents

The following procedure shows how to create a schema validation that rejects invalid documents.

Create a collection with **validationAction: "error"**.

Create a contacts collection with a JSON schema validator that has validationAction: "error":

```
db.createCollection( "contacts", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: [ "phone" ],
    properties: {
      phone: {
        bsonType: "string",
        description: "must be a string and is required"
      },
      email: {
        bsonType : "string",
        pattern : "@mongodb\\.com$",
        description: "must be a string and end with
'@mongodb.com'"
      }
    }
  } },
  validationAction: "error"
})
```

The error validationAction causes MongoDB to reject any invalid documents and prevent them from being inserted into the collection.

Attempt to insert an invalid document.

Attempt to insert the following document:

```
db.contacts.insertOne(  
  { name: "Amanda", email: "amanda@xyz.com" }  
)
```

The document violates the validation rule because:

- The email field does not match the regular expression pattern. The email field must end in @mongodb.com.
- It is missing the required phone field.

The operation fails with the following error:

MongoServerError: Document failed validation
Additional information: {
failingDocumentId: ObjectId("6377cca4aac957f2b77ea955"),
details: {
operatorName: '\$jsonSchema',
schemaRulesNotSatisfied: [
{
operatorName: 'properties',
propertiesNotSatisfied: [
{
propertyName: 'email',
description: "must be a string and end with '@mongodb.com'",
details: [
{
operatorName: 'pattern',
specifiedAs: { pattern: '@mongodb\\.com\$' },
reason: 'regular expression did not match',
consideredValue: 'amanda@xyz.com'
}
]
}
]
}
]

},
{
operatorName: 'required',
specifiedAs: { required: ['phone'] },
missingProperties: ['phone']
}
]
}
}

Option 2: Allow Invalid Documents, but Record Them in the Log

The following procedure shows how to create a schema validation that allows invalid documents, but records invalid documents in the MongoDB log.

Create a collection with **validationAction: "warn"**.

Create a contacts2 collection with a JSON schema validator that has validationAction: "warn":

```
db.createCollection( "contacts2", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: [ "phone" ],
    properties: {
      phone: {
        bsonType: "string",
        description: "must be a string and is required"
      },
      email: {
        bsonType : "string",
        pattern : "@mongodb\\.com$",
        description: "must be a string and end with
'@mongodb.com"
      }
    }
  }
}
```

```
}},  
validationAction: "warn"  
})
```

The warn validationAction allows invalid documents to be inserted into the collection. Invalid documents are recorded in the MongoDB log.

Attempt to insert an invalid document.

Attempt to insert the following document:

```
db.contacts2.insertOne(  
  { name: "Amanda", email: "amanda@xyz.com" }  
)
```

The document violates the validation rule because:

- The email field does not match the regular expression pattern. The email field must end in @mongodb.com.
- It is missing the required phone field.

Check the logs for the invalid document.

To view the MongoDB logs in a readable format, run the following command:

```
db.adminCommand(  
  { getLog:'global' } ).log.forEach(x => { print(x) }  
)
```

{
"t": {
"\$date": "2022-11-18T13:30:43.607-05:00"
},
"s": "W",
"c": "STORAGE",
"id": 20294,
"ctx": "conn2",

"msg": "Document would fail validation",
"attr": {
"namespace": "test.contacts2",
"document": {
"_id": {
"\$oid": "6377cf53d59841355cac1cd0"
},
"name": "Amanda",
"email": "amanda@xyz.com"
},
"errInfo": {
"failingDocumentId": {
"\$oid": "6377cf53d59841355cac1cd0"
},
"details": {
"operatorName": "\$jsonSchema",
"schemaRulesNotSatisfied": [{
"operatorName": "properties",
"propertiesNotSatisfied": [{
"propertyName": "email",
"description": "must be a string and end with
"details": [{
"operatorName": "pattern",
"specifiedAs": {
"pattern": "@mongodb\\.com\$"
},
"reason": "regular expression did not match",
"consideredValue": "amanda@xyz.com"
}]
}]
}, {
"operatorName": "required",
"specifiedAs": {
"required": ["phone"]
},
"missingProperties": ["phone"]

}]
}
}
}
}

Query for and Modify Valid or Invalid Documents

If you add validation to your collection after you create it, or modify an existing validation schema, you may have invalid documents in your collection. Similarly, if your schema's `validationAction` is `warn`, your collection is allowed to contain invalid documents. You can query for invalid documents to potentially update or delete them from your collection.

To find documents that either match or don't match a specified schema, use `$jsonSchema` with query operators. Similarly, you can update or delete documents based on a schema by using `$jsonSchema` in query conditions for write operations.

Examples

Create a sample collection inventory with the following documents:

```
db.inventory.insertMany( [
  { item: "journal", qty: NumberInt(25), size: { h: 14, w: 21, uom:
    "cm" }, instock: true },
  { item: "notebook", qty: NumberInt(50), size: { h: 8.5, w: 11, uom:
    "in" }, instock: true },
  { item: "paper", qty: NumberInt(100), size: { h: 8.5, w: 11, uom: "in"
  }, instock: 1 },
  { item: "planner", qty: NumberInt(75), size: { h: 22.85, w: 30, uom:
    "cm" }, instock: 1 },
  { item: "postcard", qty: NumberInt(45), size: { h: 10, w: 15.25,
    uom: "cm" }, instock: true },
```

```
{ item: "apple", qty: NumberInt(45), status: "A", instock: true },  
{ item: "pears", qty: NumberInt(50), status: "A", instock: true }  
])
```

Define a Schema Object

Define a sample schema object and store it in a variable called `myschema`:

```
let myschema =  
{  
  $jsonSchema: {  
    required: [ "item", "qty", "instock" ],  
    properties: {  
      item: { bsonType: "string" },  
      qty: { bsonType: "int" },  
      size: {  
        bsonType: "object",  
        required: [ "uom" ],  
        properties: {  
          uom: { bsonType: "string" },  
          h: { bsonType: "double" },  
          w: { bsonType: "double" }  
        }  
      },  
      instock: { bsonType: "bool" }  
    }  
  }  
}
```

Find Documents that Match the Schema

These commands return all documents that match the schema:

```
db.inventory.find(myschema)  
db.inventory.aggregate( [ { $match: myschema } ] )
```

Both commands return the same result:

[
{
_id: ObjectId("62b5cd5a14b92d148400f7a3"),
item: 'apple',
qty: 45,
status: 'A',
instock: true
},
{
_id: ObjectId("62b5cd5a14b92d148400f7a4"),
item: 'pears',
qty: 50,
status: 'A',
instock: true
}
]

Find Documents that Don't Match the Schema

To find all documents do not satisfy the schema:, use `$jsonSchema` with the `$nor` operator:

```
db.inventory.find( { $nor: [ myschema ] } )
```

OUTPUT:

[
{
_id: ObjectId("62b5cd5a14b92d148400f79e"),
item: 'journal',
qty: 25,
size: { h: 14, w: 21, uom: 'cm' },
instock: true
},
{
_id: ObjectId("62b5cd5a14b92d148400f79f"),

item: 'notebook',
qty: 50,
size: { h: 8.5, w: 11, uom: 'in' },
instock: true
},
{
_id: ObjectId("62b5cd5a14b92d148400f7a0"),
item: 'paper',
qty: 100,
size: { h: 8.5, w: 11, uom: 'in' },
instock: 1
},
{
_id: ObjectId("62b5cd5a14b92d148400f7a1"),
item: 'planner',
qty: 75,
size: { h: 22.85, w: 30, uom: 'cm' },
instock: 1
},
{
_id: ObjectId("62b5cd5a14b92d148400f7a2"),
item: 'postcard',
qty: 45,
size: { h: 10, w: 15.25, uom: 'cm' },
instock: true
}
]

Update Documents that Don't Match the Schema

This command updates all documents that don't match the schema and sets the documents' isValid field to false:

```
db.inventory.updateMany(
{
```

```

    $nor: [ myschema ]
  },
  {
    $set: { isValid: false }
  }
)

```

To verify the update, query the collection:

```
db.inventory.find()
```

[
{
_id: ObjectId("62b5cd5a14b92d148400f79e"),
item: 'journal',
qty: 25,
size: { h: 14, w: 21, uom: 'cm' },
instock: true,
isValid: false
},
{
_id: ObjectId("62b5cd5a14b92d148400f79f"),
item: 'notebook',
qty: 50,
size: { h: 8.5, w: 11, uom: 'in' },
instock: true,
isValid: false
},
{
_id: ObjectId("62b5cd5a14b92d148400f7a0"),
item: 'paper',
qty: 100,
size: { h: 8.5, w: 11, uom: 'in' },
instock: 1,
isValid: false
},
{

_id: ObjectId("62b5cd5a14b92d148400f7a1"),
item: 'planner',
qty: 75,
size: { h: 22.85, w: 30, uom: 'cm' },
instock: 1,
isValid: false
},
{
_id: ObjectId("62b5cd5a14b92d148400f7a2"),
item: 'postcard',
qty: 45,
size: { h: 10, w: 15.25, uom: 'cm' },
instock: true,
isValid: false
},
{
_id: ObjectId("62b5cd5a14b92d148400f7a3"),
item: 'apple',
qty: 45,
status: 'A',
instock: true
},
{
_id: ObjectId("62b5cd5a14b92d148400f7a4"),
item: 'pears',
qty: 50,
status: 'A',
instock: true
}
]